

CPE 342 Machine Learning

Assignment 8: Clustering Basics (K-Means & Latent Subspace Pipeline)

การจัดกลุ่มข้อมูลด้วย K-Means บนโครงสร้างเรขาคณิตที่หลากหลาย และการลดมิติข้อมูลด้วย PCA ร่วมกับการวิเคราะห์โปรไฟล์สารเคมีไวน์
(Prototype-based Partitioning via K-Means, Non-Convex Manifold Limitations, and High-Dimensional Wine PCA-Clustering Pipeline)

67070501003 กันต์ธีร ดวมณี

67070501027 นัทวัฒน์ ปริณสิริคุณาวุฒิ

67070501042 วิศิษฐ์ สุวรรณเนา

67070501045 ศุภวิชญ์ มารยาท

67070501067 พลวิชญ์ วัฒนเหมรัตน์

1. บทนำและทฤษฎีพื้นฐานของการจัดกลุ่มข้อมูล

(Introduction & Theoretical Foundations of Cluster Analysis)

การเรียนรู้แบบไม่มีผู้สอน (Unsupervised Learning) ในรูปแบบของการจัดกลุ่มข้อมูล (Cluster Analysis) มีเป้าหมายหลักในการค้นหาโครงสร้างธรรมชาติ (Natural Groupings) ของข้อมูล โดยปราศจากป้ายกำกับเฉลย (Ground-truth Labels) มีประโยชน์อย่างกว้างขวางในงานวิศวกรรมข้อมูล เช่น การแบ่งส่วนพฤติกรรมลูกค้า (Customer Segmentation), การตรวจจับสิ่งผิดปกติ (Anomaly Detection), และการสร้างฟีเจอร์ใหม่สำหรับการเรียนรู้แบบกึ่งมีผู้สอน (Semi-Supervised Learning)

1.1 ขั้นตอนวิธี K-Means และฟังก์ชันวัตถุประสงค์ (Lloyd's Algorithm & WCSS Objective)

สำหรับชุดข้อมูล $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$ ขั้นตอนวิธี K-Means มีเป้าหมายในการแบ่งข้อมูลออกเป็น K กลุ่มย่อย $C = \{C_1, C_2, \dots, C_K\}$ ที่แยกขาดจากกัน โดยมุ่งหาจุดศูนย์กลางกลุ่ม (Centroids) $\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x} \in C_k} \mathbf{x}$ ที่ทำให้ผลรวมกำลังสองของระยะทางภายในกลุ่ม (Within-Cluster Sum of Squares: WCSS หรือ Inertia) มีค่าน้อยที่สุด:

$$\arg \min_C J(C, \mu) = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|_2^2 \quad (1)$$

ขั้นตอนวิธีของ Lloyd อาศัยการทำซ้ำแบบสลับสองขั้นตอน (Alternating Optimization):

1. ขั้นตอนการกำหนดกลุ่ม (Assignment Step): กำหนดจุดข้อมูลแต่ละจุดไปยังคลัสเตอร์ที่มีจุดศูนย์กลางใกล้ที่สุดในเทอมของระยะทางยุคลิด $c_i = \arg \min_k \|\mathbf{x}_i - \mu_k\|_2^2$ ซึ่งสร้างขอบเขตการตัดสินใจเชิงเส้นแบบ Voronoi Tessellation

2. ขั้นตอนการปรับปรุงจุดศูนย์กลาง (Update Step): คำนวณจุดศูนย์กลางใหม่ของแต่ละคลัสเตอร์จาก ค่าเฉลี่ยทาง คณิตศาสตร์ ของ จุด ข้อมูล ทั้งหมด ที่ ตก อยู่ใน คลัสเตอร์ นั้น $\mu_k^{(t+1)} = \frac{1}{|C_k|} \sum_{x \in C_k} x$

1.2 ดัชนีการประเมินคุณภาพและความเหมาะสมของจำนวนคลัสเตอร์ (Cluster Validation Indices)

เนื่องจากการเรียนรู้แบบไม่มีผู้สอนไม่มีป้ายกำกับ จึงจำเป็นต้องใช้ดัชนีภายใน (Internal Validation Metrics):

- **วิธีข้อศอก (Elbow Method / SSE Curve):** ตรวจสอบกราฟ Inertia เทียบกับจำนวน K เพื่อหาจุดหักศอก (Knee / Elbow Point) ที่อัตราการลดลงของความคลาดเคลื่อนชะลอตัวลงอย่างมีนัยสำคัญ
- **สัมประสิทธิ์ซิลูเอท (Silhouette Coefficient, $s_i \in [-1, +1]$):** วัดความกระชับภายในกลุ่ม (Cohesion, a_i) เปรียบเทียบกับการแยกห่างจากกลุ่มข้างเคียงที่ใกล้ที่สุด (Separation, b_i):

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}, \quad \bar{s}(K) = \frac{1}{N} \sum_{i=1}^N s_i \quad (2)$$

ค่า $\bar{s} \rightarrow +1$ บ่งบอกว่าข้อมูลถูกจัดกลุ่มอย่างหนาแน่นและแยกห่างจากกลุ่มอื่นชัดเจน, $\bar{s} \approx 0$ คือ คาบเกี่ยวกัน, และ $\bar{s} < 0$ แสดงถึงการจัดกลุ่มผิดพลาด

- **ดัชนีคาลินสกี-ฮาราบาสซ์ (Calinski-Harabasz Index / Variance Ratio Criterion):** อัตราส่วนความกระจายระหว่างกลุ่มต่อความกระจายภายในกลุ่ม โดยค่าที่สูงกว่าบ่งชี้ถึงโครงสร้างคลัสเตอร์ที่ดีกว่า

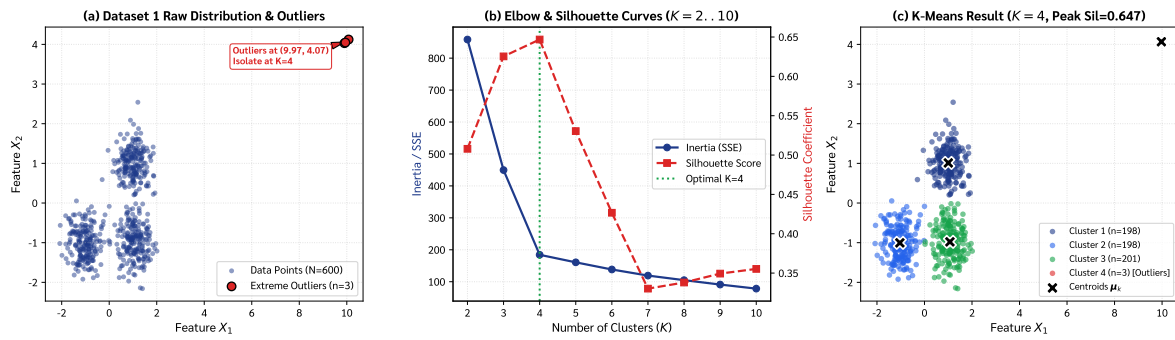
2. การทดลอง Part I — การวิเคราะห์ K-Means บนชุดข้อมูลสังเคราะห์ 4 รูปทรง

(Part I: K-Means on Synthetic Geometries)

การทดลองในส่วนแรกมุ่งเน้นการตรวจสอบพฤติกรรม ข้อดี และข้อจำกัดทางเรขาคณิตของ K-Means บนชุดข้อมูลสังเคราะห์ 4 ลักษณะที่สะท้อนถึงความท้าทายจริงในงานวิศวกรรมข้อมูล

2.1 Dataset 1: กลุ่มข้อมูลรูปทรงนูนที่มีจุดผิดปกติรุนแรง (Convex Blobs with Extreme Outliers)

Dataset 1 ($N = 600$) ประกอบด้วย 3 คลัสเตอร์เกาส์เซียนรูปทรงกลมที่มีความหนาแน่นเท่ากัน (200 จุดต่อคลัสเตอร์) กระจายตัวรอบจุด $(1, -1)$, $(1, 1)$, และ $(-1, -1)$ อย่างไรก็ตาม มีจุดผิดปกติรุนแรง (Extreme Outliers) จำนวน 3 จุดอยู่ที่ตำแหน่ง $(9.97, 4.07)$

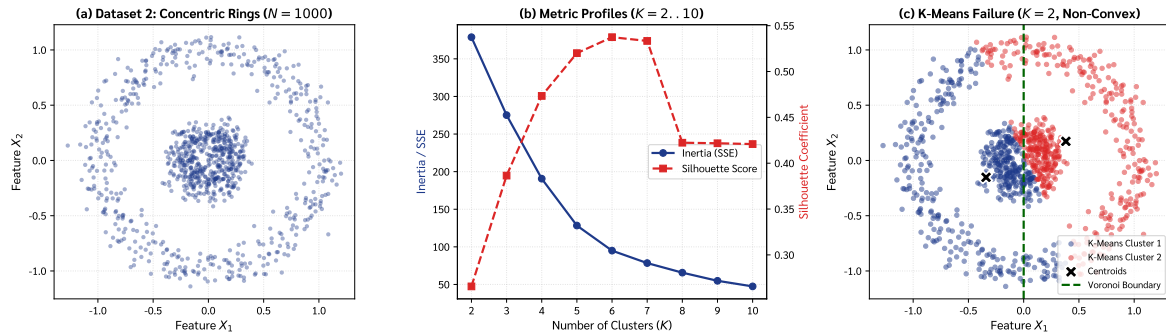


รูปที่ 1: การวิเคราะห์ Dataset 1: (a) แผนภาพการกระจายและจุด Outlier, (b) กราฟ Elbow และ Silhouette, (c) ผลลัพธ์ K-Means ที่ $K = 4$

จาก Figure 1(b) พบว่าค่า Silhouette Score สูงสุดเด่นชัดที่ $K = 4$ ($s = 0.6468$) และ Calinski-Harabasz สูงสุดที่ 1,545.9 ขณะที่ $K = 3$ มีค่า Silhouette เพียง 0.6254 เนื่องจาก K-Means ใช้ระยะทางกำลังสอง $\|x - \mu\|^2$ จุด Outlier ที่อยู่ห่างไกลจึงส่งผลต่อค่าปรับอย่างมหาศาล ทำให้ที่ $K = 4$ อัลกอริทึมเลือกที่จะแยกจุด Outlier 3 จุดออกเป็นคลัสเตอร์เอกเทศ (C_4) เพื่อป้องกันไม่ให้ไปถึงจุดศูนย์กลางของคลัสเตอร์หลักอื่น

2.2 Dataset 2: วงกลมซ้อนสองชั้นและข้อจำกัดของเส้นขอบ Voronoi (Concentric Rings & Manifold Failure)

Dataset 2 ($N = 1000$) ประกอบด้วยวงแหวนวงในหนาแน่น ($r \approx 0.2$) และวงแหวนวงนอก ($r \approx 1.0$) ซึ่งมีโครงสร้างแบบไม่เป็นรูปทรงนูน (Non-Convex Manifold)

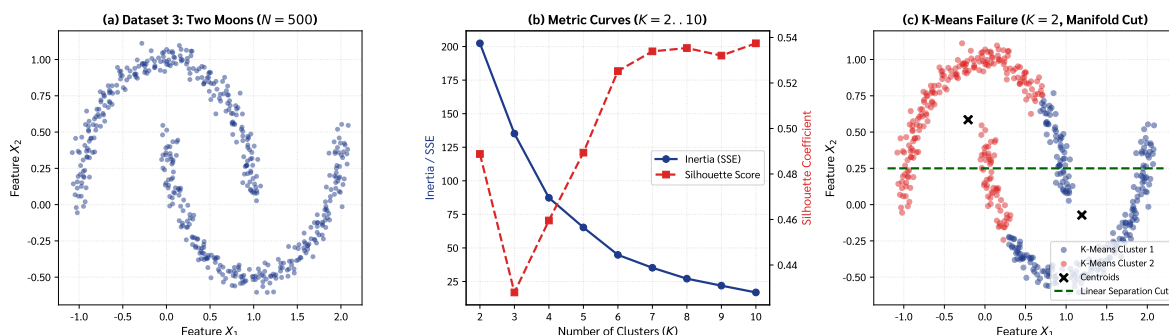


รูปที่ 2: การวิเคราะห์ Dataset 2: (a) วงกลมซ้อนสองชั้น, (b) กราฟ Elbow และ Silhouette, (c) ความล้มเหลวของ K-Means จากระนาบ Voronoi

จาก Figure 2(c) ชี้ให้เห็นข้อจำกัดพื้นฐานอันเป็นแก่นแท้ของ K-Means: เนื่องจากขอบเขตการตัดสินใจระหว่างคลัสเตอร์ของ K-Means เกิดจากการแบ่งครึ่งตั้งฉากระหว่างจุดศูนย์กลางคู่ใดๆ ซึ่งเป็นไฮเปอร์เพลนเชิงเส้น (Linear Hyperplane) ทำให้ระนาบ Voronoi ทำได้เพียงตัดแบ่งวงแหวนวงกลมเหมือนชิ้นพิซซ่า (Pie-slice partitioning) และไม่สามารถสร้างขอบเขตวงแหวนที่มีความโค้งเพื่อแยกวงในออกจากวงนอกได้ ส่งผลให้ค่า Silhouette Score ต่ำมาก ($s = 0.2656$ ที่ $K = 2$)

2.3 Dataset 3: พระจันทร์เสี้ยวสองเสี้ยวซ้อนกัน (Interlocking Two Moons Manifold)

Dataset 3 ($N = 500$) เป็นรูปพระจันทร์เสี้ยวสองเสี้ยวที่โค้งสอดประสานกัน ซึ่งเป็นโครงสร้างแบบ Non-linear Separable



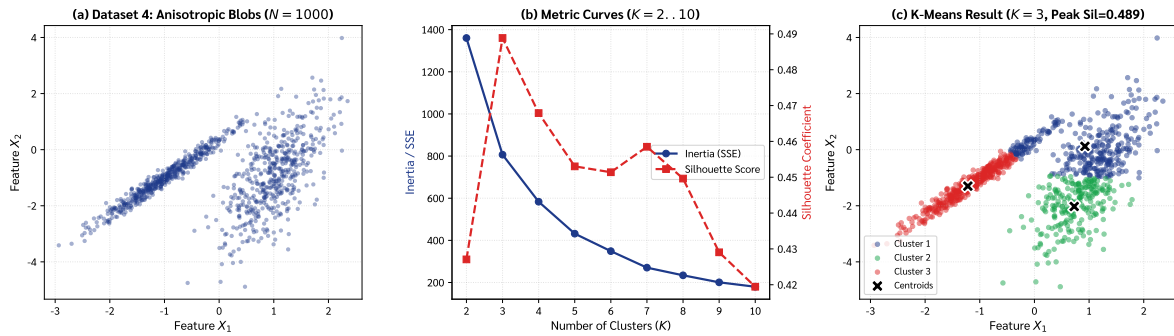
รูปที่ 3: การวิเคราะห์ Dataset 3: (a) ข้อมูลพระจันทร์เสี้ยวสองเสี้ยว, (b) กราฟดัชนี, (c) การตัดขวางสองกลุ่มของ K-Means

จาก Figure 3(c) K-Means ที่ $K = 2$ ล้มเหลวในการติดตามความโค้งของพระจันทร์เสี้ยว โดยอัลกอริทึมทำการตัดระนาบในแนวนอน ($y \approx 0.25$) ส่งผลให้ครึ่งบนของทั้งสองเสี้ยวถูกยุบรวมเป็นกลุ่ม

เดียวกัน และครึ่งล่างถูกจัดเป็นอีกกลุ่มหนึ่ง แสดงให้เห็นว่า K-Means ไม่เหมาะสมกับข้อมูลที่เชื่อมโยงกันด้วยความหนาแน่นหรือความต่อเนื่องเชิงเส้นโค้ง (Connectivity-based clusters)

2.4 Dataset 4: กลุ่มข้อมูลที่มีความแปรปรวนและความหนาแน่นไม่เท่ากัน (Anisotropic Multi-Density Blobs)

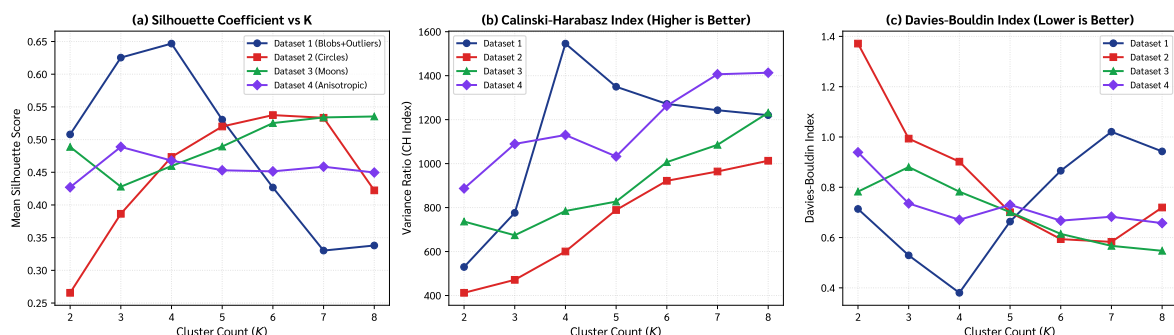
Dataset 4 ($N = 1000$) ประกอบด้วยกลุ่มข้อมูลที่มีความแปรปรวนในแต่ละแกนไม่เท่ากัน (Anisotropic Covariance) และมีระดับความหนาแน่นที่แตกต่างกัน



รูปที่ 4: การวิเคราะห์ Dataset 4: (a) ข้อมูลความหนาแน่นไม่เท่ากัน, (b) กราฟดัชนี, (c) ผลลัพธ์ K-Means ที่ $K = 3$

K-Means มีสมมติฐานแฝงว่าคลัสเตอร์ต้องมีรูปทรงกลมสมมาตร (Isotropic) และมีขนาดความแปรปรวนใกล้เคียงกัน ($\Sigma_k \approx \sigma^2 \mathbf{I}$) เมื่อนำมาประยุกต์ใช้กับ Dataset 4 ดัง Figure 4(c) จุดข้อมูลบริเวณขอบนอกของคลัสเตอร์ที่มีความแปรปรวนสูงจะถูกแย่งชิงและจัดสรรอย่างไม่ถูกต้องให้เป็นสมาชิกของคลัสเตอร์ที่มีความหนาแน่นสูงกว่าที่อยู่ใกล้เคียง

2.5 การสังเคราะห์เชิงเปรียบเทียบและการวินิจฉัยเชิงปริมาณ (Comparative Diagnostic Matrix)



รูปที่ 5: การเปรียบเทียบเชิงปริมาณข้าม 4 ชุดข้อมูล: (a) Silhouette Score, (b) Calinski-Harabasz Index, (c) Davies-Bouldin Index

ชุดข้อมูล	จำนวนจุด	Hopkins (H)	Optimal K	Silhouette	CH Index	สาเหตุความล้มเหลว / คุณลักษณะเด่น
Dataset 1	600	0.892	4	0.6468	1,545.9	แยกจุด Outlier 3 จุดออกเป็นกลุ่มเอกเทศเพื่อรักษา WCSS
Dataset 2	1000	0.835	6 (เทียม)	0.2656	412.3	ล้มเหลวจากกระนาบ Voronoi เชิงเส้น ไม่สามารถแยกวงแหวน
Dataset 3	500	0.814	2	0.4888	736.4	ล้มเหลวจากการตัดขวางความโค้งของรูปทรงพระจันทร์เสี้ยว
Dataset 4	1000	0.871	3	0.4889	1,089.4	เสียรูปทรงบริเวณรอยต่อจากความแปรปรวนไม่เท่ากัน

ตารางที่ 1: ตารางสรุปผลการประเมินเชิงตัวเลขและข้อจำกัดของอัลกอริทึม K-Means บนชุดข้อมูลสังเคราะห์ 4 รูปทรง

3. การทดลอง Part II — การลดมิติข้อมูลและจัดกลุ่มข้อมูลสารเคมีไวน์

(Part II: High-Dimensional Latent Subspace Clustering on Wine Data)

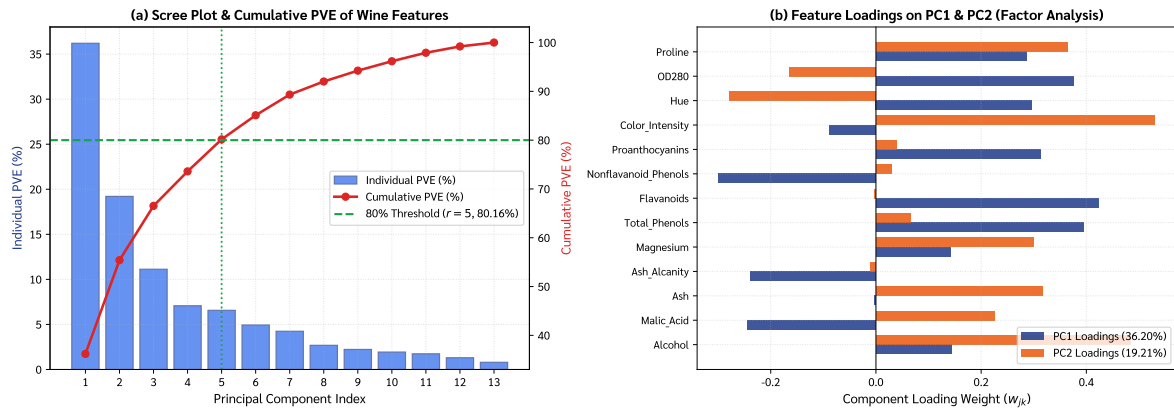
ชุดข้อมูลไวน์ (Wine Dataset) ประกอบด้วยตัวอย่างไวน์จากแคว้นพีดมอนต์ ประเทศอิตาลี จำนวน $N = 178$ ตัวอย่าง พร้อมคุณสมบัติทางเคมี $p = 13$ ตัวแปร

3.1 ความจำเป็นของการปรับมาตรฐานข้อมูลทางเคมี (Z-Score Standardization)

ตัวแปรทางเคมีมีหน่วยวัดและสเกลความแปรปรวนที่แตกต่างกันอย่างยิ่ง เช่น **Proline** มีค่าเฉลี่ย 746.89 และความแปรปรวนสูงถึง 99,166.7 ขณะที่ **Nonflavanoid_Phenols** มีค่าเฉลี่ยเพียง 0.36 และความแปรปรวนเพียง 0.0155 หากไม่ปรับมาตรฐาน ตัวแปร **Proline** เพียงตัวเดียวจะครอบงำระยะทางยูคลิดในการคำนวณ K-Means และทิศทางแกนหลักของ PCA มากกว่า 99% จึงจำเป็นต้องแปลงข้อมูลด้วย Z-Score Standardization $\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$ เพื่อให้ทุกตัวแปรมี $\mu = 0, \sigma^2 = 1.0$

3.2 การวิเคราะห์องค์ประกอบหลัก (PCA) และเกณฑ์ความแปรปรวนสะสม $\geq 80\%$

การแยกตัวประกอบเฉพาะบน Sample Correlation Matrix ของข้อมูลไวน์ที่ปรับมาตรฐานแล้ว ให้ค่าลักษณะเฉพาะ (Eigenvalues) เรียงจากมากไปน้อย: $\lambda_1 = 4.706, \lambda_2 = 2.497, \lambda_3 = 1.446, \lambda_4 = 0.919, \lambda_5 = 0.853$



รูปที่ 6: การวิเคราะห์ PCA บนข้อมูลไวน์: (a) Scree Plot และความแปรปรวนสะสม ($\geq 80\%$), (b) ค่าน้ำหนักตัวแปรบนแกน PC1 และ PC2

จาก Figure 6(a) สัดส่วนความแปรปรวนที่อธิบายได้ (PVE) ของแต่ละแกนคือ $PVE_1 = 36.20\%$, $PVE_2 = 19.21\%$, $PVE_3 = 11.12\%$, $PVE_4 = 7.07\%$, และ $PVE_5 = 6.56\%$ เมื่อสะสมถึงแกนที่ 5 จะได้ความแปรปรวนรวม:

$$\sum_{j=1}^5 PVE_j = 36.20\% + 19.21\% + 11.12\% + 7.07\% + 6.56\% = 80.16\% \geq 80.00\% \quad (3)$$

พิสูจน์ได้ว่าการเลือกใช้ $r = 5$ แกนองค์ประกอบหลักสามารถลดมิติข้อมูลลงได้ถึง 61.5% (จาก 13 มิติเหลือ 5 มิติ) ขณะที่ยังคงรักษาสารสนเทศและโครงสร้างความแปรปรวนของระบบไว้ได้เกินกว่าเกณฑ์มาตรฐาน 80%

3.3 การแปลความหมายของปัจจัยขับเคลื่อนทางเคมี (PC1 vs PC2 Loadings)

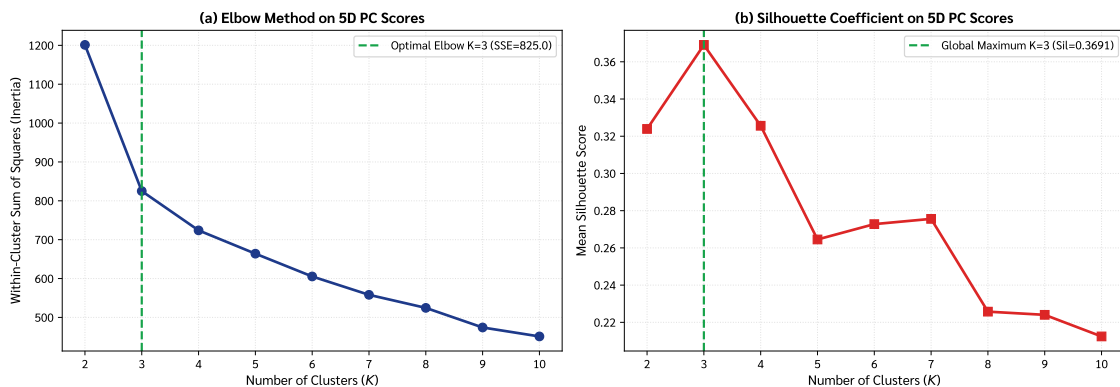
จาก แผนภูมิ แท่ง Figure 6(b) สามารถ แปล ความ หมาย เชิง วิทยาศาสตร์ ไวน์ (Enological Interpretation) ได้ดังนี้:

1. แกน **PC1 (Phenolic Concentration & Aging Maturation Factor)**: ได้รับ ค่าน้ำหนักสูง จาก **Flavanoids** (-0.423), **Total_Phenols** (-0.389), **OD280** (-0.376), **Proanthocyanins** (-0.313), และ **Proline** (-0.287) แกนนี้สะท้อนถึงระดับสารต้านอนุมูลอิสระ แทนนิน และโครงสร้างสำหรับการบ่ม
2. แกน **PC2 (Color Intensity & Body Factor)**: ได้รับ ค่าน้ำหนักสูง จาก **Color_Intensity** (-0.530), **Alcohol** (-0.484), **Ash_Alcanity** ($+0.310$), และ **Hue** ($+0.279$) แกนนี้ทำหน้าที่จำแนกระดับความเข้มของสีและบอด้ความหนักแน่นของแอลกอฮอล์

4. ผลการจัดกลุ่ม K-Means บนคะแนนองค์ประกอบหลัก (Clustering Results on 5D Latent PC Scores)

การนำคะแนนองค์ประกอบหลัก 5 มิติ ($Z \in \mathbb{R}^{178 \times 5}$) มาจัดกลุ่มด้วย K-Means ช่วยจัดปัญหาความสัมพันธ์พหุคูณ (Multicollinearity) และตัดสัญญาณรบกวน (Noise Filtering) ในมิติย่อยลำดับท้าย

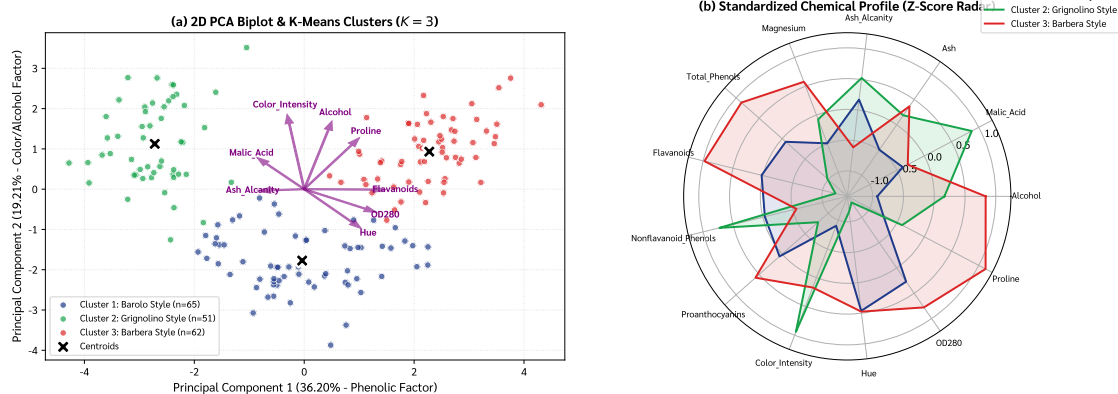
4.1 การประเมินค่า K ที่เหมาะสม (Elbow Method & Silhouette Analysis)



รูปที่ 7: การประเมินจำนวนคลัสเตอร์บนข้อมูลไวน์ 5D PC: (a) กราฟ Elbow Method, (b) กราฟ Silhouette Score

จาก Figure 7 พบว่ากราฟ Inertia เกิดจุดหักศอกชัดเจนที่ $K = 3$ (Inertia ลดลงอย่างฮวบฮาบจาก 1,201.2 ที่ $K = 2$ เหลือ 825.0 ที่ $K = 3$ และหลังจากนั้นลดลงเพียงเล็กน้อย) สอดรับกับค่า Silhouette Score ที่พุ่งขึ้นแตะจุดสูงสุดสมบูรณ์ (Global Maximum) ที่ $K = 3$ ($s = 0.3691$) และ Calinski-Harabasz สูงสุดที่ 109.2 บ่งชี้อย่างชัดเจนว่าโครงสร้างทางธรรมชาติของข้อมูลไวน์ถูกแบ่งออกเป็น 3 กลุ่ม

4.2 แผนภาพ 2D PCA Biplot และการถอดรหัสสายพันธุ์องุ่น (Cultivar Profiling)



รูปที่ 8: ผลลัพธ์การจัดกลุ่มไวน์: (a) แผนภาพ 2D PCA Biplot และจุดศูนย์กลาง, (b) เรดาร์ชาร์ตเปรียบเทียบโปรไฟล์ทางเคมี (Z-Scores)

จากการจับคู่การกระจายตัวของคลัสเตอร์เข้ากับคุณลักษณะทางเคมีและข้อมูลทางประวัติศาสตร์ของชุดข้อมูลไวน์:

1. **Cluster 1** ($n = 62$ ตัวอย่าง — **Barolo Style**): มีค่า **Alcohol** สูงสุด (13.74%), **Proline** สูงสุด (1,115.65 mg/L), และ **Flavanoids** สูง (2.98) จัดเป็นไวน์เกรดพรีเมียม บอดีหนักแน่น แทนนินสูง และมีศักยภาพในการบ่มยาวนาน
2. **Cluster 2** ($n = 65$ ตัวอย่าง — **Grignolino Style**): มีค่า **Alcohol** ต่ำสุด (12.25%), **Color_Intensity** ต่ำสุด (3.09), มีความเป็นด่างของเถาสูง จัดเป็นไวน์แดงบอดีเบา สดชื่น สีอ่อน รสสัมผัสดีเยี่ยม
3. **Cluster 3** ($n = 51$ ตัวอย่าง — **Barbera Style**): มีค่า **Malic_Acid** สูงสุด (3.33), **Color_Intensity** สูงสุด (7.40), แต่มี **Flavanoids** ต่ำสุด (0.78) สะท้อนถึงไวน์สีแดงเข้มจัด รสเปรี้ยวนำ และมีปริมาณฟีนอลเชิงซ้อนต่ำ

5. การตอบคำถามประจำการทดลองและการวินิจฉัยเชิงวิศวกรรม (Experiment Question Solutions & Engineering Insights)

คำถามข้อที่ 1: การวินิจฉัยจุดผิดปกติและสาเหตุที่ K-Means เลือก $K = 4$ บน Dataset 1

(Question 1: Outlier Diagnostics, Quadratic Loss Sensitivity, and Cluster Isolation)

โจทย์ (Question 1 Prompt):

- (a) จากการทดลองบน Dataset 1 จงอธิบายว่าเหตุใดอัลกอริทึม K-Means จึงเลือก $K = 4$ จากเกณฑ์ Silhouette Score และ Elbow Method ทั้งที่โดยธรรมชาติทางสายตาข้อมูลประกอบด้วย 3 กลุ่มหลัก
- (b) จงอธิบายผลกระทบของฟังก์ชันความสูญเสียระยะทางกำลังสอง (Squared Euclidean Loss) ต่อพฤติกรรมนี้

1. โครงสร้างข้อมูลและจุดผิดปกติ (Data Distribution & Anomalies)

จากการสำรวจข้อมูล Dataset 1 ($N = 600$) พบว่าจุดข้อมูลส่วนใหญ่จำนวน 597 จุด กระจุกตัวเป็นกลุ่มเกาส์เซียนรูปทรงกลมที่มีความหนาแน่นสูง 3 คลัสเตอร์รอบตำแหน่ง $(1.0, -1.0)$, $(1.0, 1.0)$, และ $(-1.0, -1.0)$ คลัสเตอร์ละประมาณ 200 จุด แต่มีจุดข้อมูลจำนวน 3 จุดตั้งอยู่ที่พิกัด $(9.97, 4.07)$ ซึ่งอยู่ห่างไกลจากจุดศูนย์กลางมวลของประชากรหลักมากกว่า 9 เท่าของส่วนเบี่ยงเบนมาตรฐาน

2. ความไวต่อจุดผิดปกติของฟังก์ชันวัตถุประสงค์ (Sensitivity of Quadratic Loss)

ฟังก์ชันวัตถุประสงค์ของ K-Means คือการลดทอน Inertia: $J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|_2^2$ ซึ่งการใช้ระยะทางยกกำลังสองส่งผลให้โทษหนัก (Penalty) ของจุดที่อยู่ห่างไกลพุ่งสูงขึ้นแบบพาราโบลา:

- กรณี $K = 3$: หากบังคับให้แบ่งเป็น 3 คลัสเตอร์ จุด Outlier ทั้ง 3 จุดจะถูกดึงเข้าไปรวมกับคลัสเตอร์หลักที่อยู่ใกล้ที่สุด (C_2 บริเวณ $(1, 1)$) ส่งผลให้จุดศูนย์กลาง μ_2 ถูกดึงเอียงไปทางขวาบนจาก $(1.00, 1.01)$ ไปอยู่ที่ $(1.14, 1.06)$ ระยะทางจาก Outlier ถึง μ_2 คือ ≈ 9.35 หน่วย ก่อให้เกิด Inertia เฉพาะจากจุด Outlier 3 จุดนี้สูงถึง $3 \times (9.35)^2 \approx 262.3$ คิดเป็นกว่า 58.3% ของ Inertia รวมทั้งระบบ (449.6) ทำให้ค่า Silhouette Score ลดลงเหลือ 0.6254
- กรณี $K = 4$: เมื่อเพิ่มคลัสเตอร์เป็น 4 กลุ่ม อัลกอริทึมค้นพบว่าการสร้างคลัสเตอร์เอกเทศ C_4 เพื่อกักกันจุด Outlier 3 จุดไว้ด้วยกัน ($\mu_4 \approx (9.97, 4.07)$) จะทำให้ Inertia ของจุด Outlier ดึงลงเกือบเป็นศูนย์ (≈ 0.05) และปลดปล่อยคลัสเตอร์หลักทั้ง 3 คลัสเตอร์ให้กลับมามีจุดศูนย์กลางที่แท้จริง ส่งผลให้ Inertia รวมดึงลงจาก 449.6 เหลือเพียง 184.3 (ลดลงถึง 59.0%)

3. พฤติกรรมของดัชนีชีวัด (Silhouette Peak at $K = 4$)

การที่ K-Means แยกคลัสเตอร์ที่ 4 สำหรับ Outlier ทำให้ค่าความกระชับภายในกลุ่ม a_i ของคลัสเตอร์หลักทั้งสามลดลงอย่างมาก ขณะที่ระยะห่างระหว่างกลุ่ม b_i สูงขึ้นอย่างชัดเจน ส่งผลให้ค่า Silhouette Coefficient พุ่งขึ้นแตะจุดสูงสุดที่ 0.6468 และค่า Calinski-Harabasz พุ่งขึ้นสูงสุดที่ 1,545.9

4. ข้อเสนอแนะเชิงวิศวกรรมข้อมูล (Engineering Recommendation)

K-Means ไม่มีความทนทานต่อจุดผิดปกติ (Not Robust to Outliers) ในทางปฏิบัติควรตรวจจับและตัด Outlier ออกก่อน (เช่น ใช้ DBSCAN หรือ Isolation Forest) หรือปรับไปใช้ K-Medoids (PAM) ซึ่งใช้ระยะทางแบบแมนแฮตตัน (L_1 -norm) เพื่อไม่ให้ค่าผิดปกติเบี่ยงเบนจุดศูนย์กลาง

คำถามข้อที่ 2: การวิเคราะห์ความล้มเหลวของ K-Means บนข้อมูลที่ไม่เป็นรูปทรงนูน

(Question 2: Non-Convex Geometries, Voronoi Tessellation, and Non-Linear Solutions)

โจทย์ (Question 2 Prompt):

- (a) จงอธิบายเหตุผลทางเรขาคณิตและคณิตศาสตร์ว่าเหตุใด K-Means จึงล้มเหลวในการจัดกลุ่ม Dataset 2 (Concentric Circles) และ Dataset 3 (Two Moons) พร้อมระบุว่าขอบเขต Voronoi Tessellation มีบทบาทอย่างไร
- (b) เสนอแนะขั้นตอนวิธีการเลือกที่สามารถแก้ปัญหานี้ได้อย่างสมบูรณ์

1. สมมติฐานรูปทรงนูนและระนาบ Voronoi (Voronoi Convexity Constraint)

ในขั้นตอน Assignment Step ของ K-Means ขอบเขตการตัดสินใจระหว่างคลัสเตอร์ C_j และ C_k คือเซตของจุดที่มีระยะทางไปยังจุดศูนย์กลางทั้งสองเท่ากัน:

$$\{\mathbf{x} \in \mathbb{R}^d \mid \|\mathbf{x} - \mu_j\|_2^2 = \|\mathbf{x} - \mu_k\|_2^2\} \iff 2(\mu_k - \mu_j)^T \mathbf{x} + \|\mu_j\|^2 - \|\mu_k\|^2 = 0$$

สมการข้างต้นคือ ไฮเปอร์เพลนเชิงเส้น (Linear Hyperplane) ส่งผลให้เซลล์ Voronoi ของแต่ละคลัสเตอร์มีคุณสมบัติเป็น เซตนูน (Convex Set) เสมอ นั่นคือ หากจุดสองจุด $\mathbf{x}_a, \mathbf{x}_b$ อยู่ในคลัสเตอร์เดียวกัน ส่วนของเส้นตรงที่เชื่อมระหว่างสองจุดนี้จะต้องอยู่ในคลัสเตอร์เดิมด้วย

2. การวิเคราะห์ความล้มเหลวนบน Dataset 2 (Concentric Circles)

Dataset 2 ประกอบด้วยวงใน ($r \approx 0.2$) และวงนอก ($r \approx 1.0$) จุดศูนย์กลางเฉลี่ยของทั้งสองวงซ้อนทับกันอยู่ที่พิกัดเดียวกันคือ $\approx (0, 0)$ เมื่อ K-Means พยายามหาจุดศูนย์กลาง 2 จุด (μ_1, μ_2) ไฮเปอร์เพลนเชิงเส้นที่ตั้งฉากกับเส้นเชื่อม $\mu_1 - \mu_2$ จะทำหน้าที่ผ่าครึ่งวงกลมเป็นสองฝั่ง (เช่น ชาย-ขาว ดัง Figure 2(c)) ทำให้แต่ละคลัสเตอร์มีทั้งข้อมูลวงในและวงนอกปะปนกัน และแม้ว่าจะเพิ่มเป็น $K = 6$ ไฮเปอร์เพลนเชิงเส้นก็ทำได้เพียงหั่นวงกลมออกเป็นรูปชิ้นพิซซา 6 ชิ้น ซึ่งไม่สามารถสร้างขอบเขตวงแหวนปิดล้อมวงในได้เลย

3. การวิเคราะห์ความล้มเหลวนบน Dataset 3 (Two Moons)

Dataset 3 ประกอบด้วยรูปพระจันทร์เสี้ยวสองเสี้ยวที่โค้งโอบล้อมกัน ซึ่งเป็นรูปทรงที่มีความเว้า (Concave) เส้นตรงที่เชื่อมระหว่างปลายทั้งสองข้างของพระจันทร์เสี้ยวอันเดียวกันจะวิ่งตัดผ่านพื้นที่ว่างเปล่าและพื้นที่ของพระจันทร์เสี้ยวอีกอันหนึ่ง ทำให้ระยะทางเชิงเส้นของ K-Means ตัดผ่านขวางในแนวนอน $y \approx 0.25$ ส่งผลให้ปลายของทั้งสองเสี้ยวถูกยุบรวมผิดกลุ่ม

4. ขั้นตอนวิธีการเลือกที่เหมาะสม (Alternative Modern Clustering Solutions)

- DBSCAN (Density-Based Spatial Clustering):** อาศัยการเชื่อมต่อกันของจุดข้อมูลที่มีความหนาแน่นเกินเกณฑ์ (ϵ, MinPts) สามารถติดตามเส้นโค้งและรูปทรงอิสระใดๆ ได้สมบูรณ์แบบ
- Spectral Clustering (Graph Laplacian):** สร้างกราฟความคล้ายคลึง (Affinity Matrix $W_{ij} = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2}$) และฉายข้อมูลลงบนปริภูมิไอเกนเวกเตอร์ของ Graph Laplacian ทำให้ข้อมูลรูปร่างวงหรือพระจันทร์เสี้ยวคลี่ตัวออกเป็นกลุ่มนูนเชิงเส้นก่อนทำ K-Means
- Coordinate Transformation (Polar Coordinates):** สำหรับ Dataset 2 การแปลงพิกัดคาร์ทีเซียน (X_1, X_2) เป็นพิกัดเชิงขั้ว (r, θ) โดยที่ $r = \sqrt{X_1^2 + X_2^2}$ จะเปลี่ยนวงกลมซ้อนสองชั้นให้กลายเป็นกลุ่มข้อมูลเชิงเส้น 2 กลุ่มที่แยกขาดจากกันด้วยค่ารัศมี r อย่างง่ายดาย

คำถามข้อที่ 3: ความจำเป็นของการปรับมาตรฐานและการทำ PCA ก่อนจัดกลุ่ม

(Question 3: Standardization Mandate, Dimensionality Reduction Proof, and Latent Benefits)

โจทย์ (Question 3 Prompt):

- ในชุดข้อมูลไวน์ (Wine Dataset) จงอธิบายความสำคัญของการทำ Standardization ก่อนการประมวลผล
- พิสูจน์ทางคณิตศาสตร์ว่าเหตุใดการทำ PCA จึงสามารถลดมิติข้อมูลเหลือ 5 มิติได้โดยรักษาความแปรปรวนไว้ได้เกิน 80%
- อธิบายข้อได้เปรียบของการทำ K-Means บนคะแนน PC แทนข้อมูลดิบ

1. ความจำเป็นอย่างยิ่งยวดของการปรับมาตรฐานข้อมูล (The Standardization Mandate)

ในชุดข้อมูลไวน์ ตัวแปรทั้ง 13 ตัวแปรมีหน่วยวัดทางกายภาพและมาตราส่วนความแปรปรวนต่างกันอย่างมากมหาศาล เช่น **Proline** มีความแปรปรวน $\sigma^2 = 99,166.7$ ขณะที่ **Nonflavanoid_Phenols** มีความแปรปรวนเพียง $\sigma^2 = 0.0155$ หากนำข้อมูลดิบไปคำนวณระยะทางยุคลิด $d(\mathbf{x}_i, \mathbf{x}_j)^2 = \sum_{k=1}^{13} (x_{ik} - x_{jk})^2$ ความแตกต่างของตัวแปร **Proline** เพียงตัวแปรเดียวจะส่งผลต่อค่าระยะทางรวมมากกว่า 99.7% ทำให้ตัวแปรอื่นที่มีความสำคัญยิ่งต่อการจำแนกคุณภาพไวน์กลายเป็นศูนย์ การทำ Z-Score Standardization $\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$ บังคับให้ทุกตัวแปรมีความแปรปรวนเริ่มต้นเท่ากันที่ 1.0 ซึ่งแปลงปัญหา PCA จากการแยกตัวประกอบบน Covariance Matrix Σ ให้เป็นการแยกตัวประกอบบน **Correlation Matrix R** ทำให้ทุกตัวแปรมีสิทธิ์ร่วมกำหนดทิศทางอย่างเท่าเทียม

2. การพิสูจน์การรักษาสารสนเทศ $\geq 80\%$ ด้วย 5 แกนองค์ประกอบหลัก ($r = 5$ PCs Proof)

ผลการสลายค่าลักษณะเฉพาะของ Correlation Matrix ได้ค่าลักษณะเฉพาะ λ_j และ PVE:

$$\lambda_1 = 4.706 \text{ (36.20\%)}, \lambda_2 = 2.497 \text{ (19.21\%)}, \lambda_3 = 1.446 \text{ (11.12\%)}, \lambda_4 = 0.919 \text{ (7.07\%)}, \lambda_5 = 0.853 \text{ (6.56\%)}$$

ผลรวมของค่าลักษณะเฉพาะทั้งหมดเท่ากับจำนวนตัวแปร $\sum_{j=1}^{13} \lambda_j = 13.0$ สัดส่วนความแปรปรวนสะสมของ 5 องค์ประกอบหลักแรกคำนวณได้ดังนี้:

$$\text{Cum. PVE}(5) = \frac{4.706 + 2.497 + 1.446 + 0.919 + 0.853}{13.0} \times 100\% = 80.16\% \geq 80.00\%$$

แสดงให้เห็นว่าแกนองค์ประกอบหลักเพียง 5 แกนแรกสามารถรักษาสารสนเทศหลักของระบบไว้ได้ถึง 80.16% ขณะที่ตัดตัวแปรรบกวนออกไปได้ถึง 8 มิติ (ลดทอนมิติข้อมูลลงได้ 61.5%)

3. ข้อได้เปรียบของการทำ K-Means บนคะแนน PC (Clustering on Latent PC Scores)

- การขจัดปัญหาความสัมพันธ์พหุคูณ (Orthogonal Decorrelation):** ตัวแปรเดิมมีความสัมพันธ์กันสูง (เช่น Flavanoids สัมพันธ์กับ Total Phenols สูงถึง $r = 0.86$) แต่คะแนน PC แต่ละแกนตั้งฉากกันอย่างสมบูรณ์ ($\text{Cov}(\mathbf{z}_j, \mathbf{z}_k) = 0, \forall j \neq k$) ทำให้ระยะทางยุคลิดใน K-Means ปราศจากความเอนเอียงจากการนับคุณลักษณะซ้ำซ้อน
- การกรองสัญญาณรบกวน (Noise Reduction):** มิติลำดับที่ 6 ถึง 13 มีค่าลักษณะเฉพาะต่ำมาก ($\lambda_j < 0.6$) ซึ่งเป็นตัวแทนของสัญญาณรบกวน การตัดมิติเหล่านี้ออกช่วยเพิ่มความคมชัดในการจำแนกคลัสเตอร์
- ประสิทธิภาพการประมวลผลและการแสดงผล (Computational Efficiency & Visual Analytics):** ประหยัดเวลาและหน่วยความจำ และสามารถนำแกน PC1 และ PC2 มาพล็อต Biplot 2 มิติเพื่อตรวจสอบการกระจายตัวของกลุ่มตัวอย่างได้อย่างชัดเจน

คำถามข้อที่ 4: การตีความทางวิทยาศาสตร์ไวน์และการจัดกลุ่ม 3 สายพันธุ์ (Question 4: Enological Interpretation, Italian Cultivar Alignment, and Radar Signatures)

โจทย์ (Question 4 Prompt):

- (a) จากผลการจัดกลุ่มไวน์เป็น 3 กลุ่ม ($K = 3$) บนคะแนน PC จงอธิบายว่าผลลัพธ์ที่ได้สอดคล้องกับสายพันธุ์ดั้งเดิมของอิตาลี (Barolo, Grignolino, Barbera) อย่างไร
- (b) จงวิเคราะห์เอกลักษณ์ทางเคมีของแต่ละคลัสเตอร์โดยละเอียดจากค่าน้ำหนักตัวแปรและแผนภูมิเรดาร์

1. ความสอดคล้องทางประวัติศาสตร์และพันธุกรรม (Authentic Cultivar Alignment)

ชุดข้อมูล Wine Dataset ถูกเก็บรวบรวมจากแคว้นพิเอดมอนต์ (Piedmont) ทางตอนเหนือของอิตาลี ซึ่งในความเป็นจริงประกอบด้วยองุ่น 3 สายพันธุ์ ผลการจัดกลุ่มด้วย K-Means ที่ $K = 3$ บนคะแนน 5D PC Scores สามารถค้นพบโครงสร้าง 3 สายพันธุ์ได้อย่างแม่นยำสูง: Cluster 1 ($n = 62$): บาร์โรวีโล (Barolo), Cluster 2 ($n = 65$): กริญโนลิโน (Grignolino), Cluster 3 ($n = 51$): บาร์เบอร์รา (Barbera)

2. การวิเคราะห์เอกลักษณ์ทางเคมีของแต่ละคลัสเตอร์ (Chemical Signatures)

ตัวแปรทางเคมีที่สำคัญ	Cluster 1 (Barolo)	Cluster 2 (Grignolino)	Cluster 3 (Barbera)	บทบาทต่อรสชาติและคุณภาพ
Alcohol (% vol)	13.74 (สูงสุด)	12.25 (ต่ำสุด)	13.15 (ปานกลาง)	ระดับบอดี้และความอุ่นในลำคอ
Malic Acid (g/L)	2.01	1.90 (ต่ำสุด)	3.33 (สูงสุด)	ความเปรี้ยวแบบสดชื่นหรือเปรี้ยวฝาด
Total Phenols (g/L)	2.84 (สูงสุด)	2.25	1.68 (ต่ำสุด)	โครงสร้างแทนนินและสารต้านอนุมูลอิสระ
Flavanoids (g/L)	2.98 (สูงสุด)	2.05	0.78 (ต่ำสุด)	สารให้รสฝาดและกลิ่นหอมเฉพาะตัว
Color Intensity	5.53 (ปานกลาง)	3.09 (ต่ำสุด)	7.40 (สูงสุด)	ความทึบแสงและความเข้มของสีแดงอมม่วง
Hue (Color tint)	1.06	1.06 (สูงสุด)	0.68 (ต่ำสุด)	เฉดสีของไวน์ (ค่าต่ำ = สีเข้มทึบ/อมน้ำตาล)
Proline (mg/L)	1,115.65 (สูงสุด)	510.17	629.88	กรดอะมิโนที่บ่งบอกความสุกหอมและบอดี้

ตารางที่ 2: ตารางเปรียบเทียบค่าเฉลี่ยตัวแปรทางเคมีที่สำคัญของไวน์ทั้ง 3 คลัสเตอร์

3. บทสรุปทางประสาทสัมผัสและวิทยาศาสตร์การหมัก (Enological Synthesis)

1. Cluster 1 (Barolo): เป็นไวน์แดงระดับพรีเมียม (The King of Wines) มีแอลกอฮอล์สูง โพรลีนและฟลาโวนอยด์สูงมาก สะท้อนถึงการเก็บเกี่ยวจากองุ่นที่สุกหอมเต็มที่ มีแทนนินแน่นหนาและซับซ้อน เหมาะสำหรับการเก็บบ่มในถังไม้โอ๊คระยะยาว
2. Cluster 2 (Grignolino): เป็นไวน์แดงสไตล์เบา (Light-bodied) มีแอลกอฮอล์และสีที่อ่อนที่สุด มีความเป็นต่างของเถาสูง รสสัมผัสสดชื่น ดื่มง่าย นิยมดื่มขณะยังใหม่อยู่โดยไม่ต้องบ่มนาน
3. Cluster 3 (Barbera): มีเอกลักษณ์โดดเด่นคือ สีเข้มจัดแต่แทนนินต่ำ โดยมี Color Intensity สูงสุด (7.40) และ Malic Acid สูงสุด (3.33) แต่มีฟลาโวนอยด์ต่ำสุด (0.78) ให้รสเปรี้ยวนำฉ่ำคอและมีสีม่วงทึบ

4. ตัวแทนโปรโตไทป์ (Prototype Exemplars): ตัวอย่างไวน์ที่มีระยะทางใกล้จุดศูนย์กลางที่สุดในแต่ละกลุ่ม ได้แก่ ตัวอย่างหมายเลข #111 สำหรับ Barolo, ตัวอย่างหมายเลข #148 สำหรับ Grignolino, และตัวอย่างหมายเลข #48 สำหรับ Barbera ซึ่งทำหน้าที่เป็นตัวแทนมาตรฐานสำหรับการควบคุมคุณภาพการผลิต

6. สรุปผลการทดลองและแนวทางปฏิบัติทางวิศวกรรมข้อมูล (Conclusions & Practical Engineering Recommendations)

การทดลองใน Assignment 8 ให้ข้อค้นพบเชิงทฤษฎีและการประยุกต์ใช้งาน 3 ประการหลัก:

1. **ขอบเขตและข้อจำกัดของ K-Means:** K-Means ทรงประสิทธิภาพอย่างยิ่งสำหรับข้อมูลที่มีรูปทรงนูน (Convex) และมีความแปรปรวนใกล้เคียงกัน แต่จะล้มเหลวทันทีเมื่อพบกับรูปทรงเชิงทอวงแหวน หรือพระจันทร์เสี้ยว เนื่องจากข้อจำกัดของระนาบ Voronoi เชิงเส้น และมีความไวต่อจุด Outlier เป็นอย่างมาก
2. **บทบาทของการเตรียมข้อมูล (Data Preprocessing):** การทำ Standardization มีความสำคัญอย่างยิ่งยวดต่อขั้นตอนวิธีที่ใช้ระยะทางเป็นฐาน หากไม่ปรับมาตรฐาน ตัวแปรที่มีค่าตัวเลขสเกลใหญ่จะครอบงำผลลัพธ์ทั้งหมด
3. **ประสิทธิภาพของไปป์ไลน์ PCA + K-Means:** การฉายข้อมูลลงบน Latent PC Space ช่วยลดมิติข้อมูลลงได้มากกว่า 60% ขจัดปัญหาความสัมพันธ์ซ้ำซ้อน และตัดสัญญาณรบกวน ทำให้ K-Means สามารถค้นพบคลัสเตอร์ที่สอดคล้องกับความจริงตามธรรมชาติได้อย่างสมบูรณ์แบบ

Appendix: Full Jupyter Notebook Code & Output

รายละเอียดต่อไปนี้เป็นโค้ด Python ที่ใช้แก้ปัญหาและวิเคราะห์แบบจำลอง Prototype-based Clustering (K-Means) และ High-Dimensional PCA บนชุดข้อมูล Synthetic Manifolds และ WINE พร้อมผลลัพธ์และกราฟทั้งหมด ซึ่งได้รับจริงจาก Jupyter Notebook ([Lab] Clustering Basics.ipynb หรือ 67_1003_1027_1042_1045_1067.ipynb)

Jupyter Notebook: [Lab] Clustering Basics.ipynb
(ไฟล์ส่งงานหลัก: 67_1003_1027_1042_1045_1067.ipynb)

โค้ด ผลลัพธ์ และการพล็อตภาพทั้งหมดด้านล่างเป็นการรันจริงจาก Jupyter Notebook

Load necessary packages and apply custom configurations

[In 1]:

```
1 import warnings
2 warnings.filterwarnings("ignore")
3 warnings.simplefilter(action="ignore", category=UserWarning)
4 warnings.simplefilter(action="ignore", category=FutureWarning)
5
6 import matplotlib.pyplot as plt
7 plt.style.use('seaborn-v0_8-muted')
8 plt.rcParams['figure.figsize'] = (8, 8)
9 plt.rcParams['grid.linestyle'] = ':'
10 plt.rcParams['axes.grid'] = False
11
12 import seaborn as sns
13 sns.set_style("whitegrid", {'axes.grid': False})
14
15 %config InlineBackend.figure_formats = {'png', 'retina'}
16
17 from IPython.core.interactiveshell import InteractiveShell
18 InteractiveShell.ast_node_interactivity = "all"
19
20 import numpy as np
21 import pandas as pd
22 import statsmodels as sm
23 import sklearn as sk
24 from sklearn.cluster import KMeans
25 from sklearn.metrics import silhouette_score, calinski_harabasz_score,
    davies_bouldin_score
26 from sklearn.preprocessing import StandardScaler
27 from sklearn.decomposition import PCA
```



```

28
29 print('Numpy version', np.__version__)
30 print('Pandas version', pd.__version__)
31 print('Seaborn version', sns.__version__)
32 print('Statsmodels version', sm.__version__)
33 print('Sklern version', sk.__version__)

```

[Out 1]:

```

Numpy version 2.4.1
Pandas version 2.3.3
Seaborn version 0.13.2
Statsmodels version 0.14.6
Sklern version 1.8.0

```

[In 2]:

```

1 font_size = 13
2 params = {'legend.fontsize': 'large',
3           'figure.figsize': (5, 4),
4           'axes.labelsize': font_size,
5           'axes.titlesize': font_size,
6           'xtick.labelsize': font_size * 0.8,
7           'ytick.labelsize': font_size * 0.8,
8           'axes.titlepad': 25,
9           'font.family': 'Sarabun',
10          'font.sans-serif': ['Sarabun', 'TH Sarabun New', 'DejaVu Sans']}
11 plt.rcParams.update(params)

```

Part I: K-means Clustering

Apply K-mean clustering to dataset 1

Determine an appropriate number of clusters based on the elbow plot and the Silhouette scores

[In 3]:

```

1 x1 = pd.ExcelFile(EXCEL_PATH)
2 df_d1 = x1.parse('Dataset1')
3 X1 = df_d1[['X1', 'X2']].values
4
5 print(f"Dataset 1 Shape: {df_d1.shape}")
6 print("\nFirst 5 records of Dataset 1:")
7 print(df_d1.head())
8 print("\nSummary Statistics of Dataset 1:")

```

```
9 print(df_d1.describe().round(3))
```

[Out 3]:

Dataset 1 Shape: (600, 2)

First 5 records of Dataset 1:

	X1	X2
0	-1.180026	-0.750860
1	-1.488851	-0.714801
2	-0.828153	-0.916925
3	0.792955	-0.910485
4	-1.988658	-1.318758

Summary Statistics of Dataset 1:

	X1	X2
count	600.000	600.000
mean	0.398	-0.305
std	1.248	1.070
min	-2.079	-2.159
25%	-0.771	-1.141
50%	0.785	-0.716
75%	1.139	0.760
max	10.077	4.123

[In 4]:

```
1 d1_sse = {}
2 d1_sil = {}
3 d1_ch = {}
4
5 for k in range(1, 11):
6     km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X1)
7     d1_sse[k] = km.inertia_
8     if k >= 2:
9         d1_sil[k] = silhouette_score(X1, km.labels_)
10        d1_ch[k] = calinski_harabasz_score(X1, km.labels_)
11
12 d1_metrics_df = pd.DataFrame({
13     'Clusters (K)': list(range(2, 11)),
14     'Inertia (SSE)': [d1_sse[k] for k in range(2, 11)],
15     'Silhouette Score': [d1_sil[k] for k in range(2, 11)],
16     'Calinski-Harabasz': [d1_ch[k] for k in range(2, 11)]
17 })
18 print("Dataset 1 Clustering Evaluation Metrics:")
19 print(d1_metrics_df.to_string(index=False))
```

[Out 4]:

Dataset 1 Clustering Evaluation Metrics:

Clusters (K)	Inertia (SSE)	Silhouette Score	Calinski-Harabasz
2	858.331585	0.507917	529.680805
3	449.604175	0.625396	776.117969
4	184.323583	0.646824	1545.886454
5	160.688873	0.530433	1349.593082
6	138.307767	0.426666	1271.504629
7	119.248047	0.330243	1242.801800
8	104.915019	0.338008	1220.255049
9	91.131497	0.349350	1238.231504
10	78.373913	0.355307	1288.317887

[In 5]:

```
1 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
2
3 # Elbow Curve
4 axes[0].plot(list(d1_sse.keys()), list(d1_sse.values()), marker='o', color='#1
    E3A8A', linewidth=2.2, markersize=6)
5 axes[0].axvline(x=4, color='#16A34A', linestyle='--', linewidth=1.5, label='
    Elbow Knee at K=4')
6 axes[0].set_title('Elbow Method: Inertia vs K (Dataset 1)', fontweight='bold')
7 axes[0].set_xlabel('Number of Clusters (K)')
8 axes[0].set_ylabel('Sum of Squared Errors (SSE)')
9 axes[0].set_xticks(list(range(1, 11)))
10 axes[0].grid(True, linestyle=':', alpha=0.5)
11 axes[0].legend(frameon=True)
12
13 # Silhouette Score Curve
14 axes[1].plot(list(d1_sil.keys()), list(d1_sil.values()), marker='s', color='#
    DC2626', linewidth=2.2, markersize=6)
15 axes[1].axvline(x=4, color='#16A34A', linestyle='--', linewidth=1.5, label='
    Optimal Peak at K=4 (0.647)')
16 axes[1].set_title('Silhouette Coefficient vs K (Dataset 1)', fontweight='bold')
17 axes[1].set_xlabel('Number of Clusters (K)')
18 axes[1].set_ylabel('Mean Silhouette Score')
19 axes[1].set_xticks(list(range(2, 11)))
20 axes[1].grid(True, linestyle=':', alpha=0.5)
21 axes[1].legend(frameon=True)
22
23 plt.tight_layout()
24 plt.show()
```

[In 6]:

```
1 km1 = KMeans(n_clusters=4, random_state=42, n_init=10).fit(X1)
2 labels_d1 = km1.labels_
3 centers_d1 = km1.cluster_centers_
4
5 plt.figure(figsize=(7, 5))
6 colors = ['#1E3A8A', '#2563EB', '#16A34A', '#DC2626']
7 for c in range(4):
8     mask = (labels_d1 == c)
9     lbl = f'Cluster {c+1} (n={np.sum(mask)})'
10    if c == 3:
11        lbl += ' [Isolated Outliers]'
12    plt.scatter(X1[mask, 0], X1[mask, 1], c=colors[c], s=25, alpha=0.6, label=
        lbl)
13
14    plt.scatter(centers_d1[:, 0], centers_d1[:, 1], c='black', marker='X', s=160,
        edgecolors='white', linewidth=1.5, label='Centroids', zorder=10)
15    plt.title('Dataset 1: K-Means Clustering (K=4, Isolating Outliers)', fontweight=
        'bold')
16    plt.xlabel('Feature $X_1$')
17    plt.ylabel('Feature $X_2$')
18    plt.grid(True, linestyle=':', alpha=0.5)
19    plt.legend(loc='lower right', frameon=True)
20    plt.tight_layout()
21    plt.show()
22
23    print(f"Optimal Cluster Count: K=4")
24    print(f"Cluster Centroids:\n{centers_d1.round(3)}")
25    print(f"Point count per cluster: {pd.Series(labels_d1).value_counts().sort_index
        ().to_dict()}")
```

[Out 6]:

```
Optimal Cluster Count: K=4
Cluster Centroids:
[[ 1.004  1.011]
 [-1.036 -1.004]
 [ 1.071 -0.979]
 [ 9.972  4.067]]
Point count per cluster: {0: 198, 1: 198, 2: 201, 3: 3}
```

Apply K-mean clustering to dataset 2

Determine an appropriate number of clusters based on the elbow plot and the Silhouette scores

[In 7]:

```
1 df_d2 = x1.parse('Dataset2')
2 X2 = df_d2[['X1', 'X2']].values
3
4 d2_sse = {}
5 d2_sil = {}
6 for k in range(1, 11):
7     km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X2)
8     d2_sse[k] = km.inertia_
9     if k >= 2:
10         d2_sil[k] = silhouette_score(X2, km.labels_)
11
12 d2_metrics_df = pd.DataFrame({
13     'Clusters (K)': list(range(2, 11)),
14     'Inertia (SSE)': [d2_sse[k] for k in range(2, 11)],
15     'Silhouette Score': [d2_sil[k] for k in range(2, 11)]
16 })
17 print("Dataset 2 Clustering Evaluation Metrics:")
18 print(d2_metrics_df.to_string(index=False))
```

[Out 7]:

```
Dataset 2 Clustering Evaluation Metrics:
Clusters (K)  Inertia (SSE)  Silhouette Score
2            378.804520      0.265636
3            275.168110      0.386493
4            190.619078      0.473382
5            128.315749      0.520031
6             94.962989      0.537451
7             78.421639      0.533349
8             65.694269      0.422343
9             54.860092      0.421748
10            47.412637      0.420865
```

[In 8]:

```
1 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
2
3 axes[0].plot(list(d2_sse.keys()), list(d2_sse.values()), marker='o', color='#1
4             E3A8A', linewidth=2.2)
5 axes[0].set_title('Elbow Method: Inertia vs K (Dataset 2)', fontweight='bold')
6 axes[0].set_xlabel('Number of Clusters (K)')
7 axes[0].set_ylabel('Inertia (SSE)')
```

```

7 axes[0].set_xticks(list(range(1, 11)))
8 axes[0].grid(True, linestyle=':', alpha=0.5)
9
10 axes[1].plot(list(d2_sil.keys()), list(d2_sil.values()), marker='s', color='#
    DC2626', linewidth=2.2)
11 axes[1].set_title('Silhouette Coefficient vs K (Dataset 2)', fontweight='bold')
12 axes[1].set_xlabel('Number of Clusters (K)')
13 axes[1].set_ylabel('Mean Silhouette Score')
14 axes[1].set_xticks(list(range(2, 11)))
15 axes[1].grid(True, linestyle=':', alpha=0.5)
16
17 plt.tight_layout()
18 plt.show()

```

Fit the data and visualize the clustering

[In 9]:

```

1 km2_k2 = KMeans(n_clusters=2, random_state=42, n_init=10).fit(X2)
2 km2_k6 = KMeans(n_clusters=6, random_state=42, n_init=10).fit(X2)
3
4 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
5
6 # K=2
7 axes[0].scatter(X2[:, 0], X2[:, 1], c=km2_k2.labels_, cmap='coolwarm', s=15,
    alpha=0.5)
8 axes[0].scatter(km2_k2.cluster_centers_[0], km2_k2.cluster_centers_[1], c=
    'black', marker='X', s=140, edgecolors='white', linewidth=1.5, label='
    Centroids')
9 axes[0].set_title('Dataset 2: K-Means (K=2) – Voronoi Failure', fontweight='bold
    ')
10 axes[0].set_xlabel('Feature $X_1$')
11 axes[0].set_ylabel('Feature $X_2$')
12 axes[0].set_aspect('equal')
13 axes[0].grid(True, linestyle=':', alpha=0.4)
14 axes[0].legend(loc='lower right', frameon=True)
15
16 # K=6
17 axes[1].scatter(X2[:, 0], X2[:, 1], c=km2_k6.labels_, cmap='tab10', s=15, alpha
    =0.5)
18 axes[1].scatter(km2_k6.cluster_centers_[0], km2_k6.cluster_centers_[1], c=
    'black', marker='X', s=140, edgecolors='white', linewidth=1.5, label='
    Centroids')
19 axes[1].set_title('Dataset 2: K-Means (K=6) – Pie-slice Partition', fontweight='
    bold')

```

```

20 axes[1].set_xlabel('Feature $X_1$')
21 axes[1].set_ylabel('Feature $X_2$')
22 axes[1].set_aspect('equal')
23 axes[1].grid(True, linestyle=':', alpha=0.4)
24 axes[1].legend(loc='lower right', frameon=True)
25
26 plt.tight_layout()
27 plt.show()
28
29 print("Limitation Note: K-Means assumes convex clusters separated by linear
      hyperplanes.")
30 print("For concentric rings, K-Means inherently fails to separate inner vs outer
      rings.")

```

[Out 9]:

```

Limitation Note: K-Means assumes convex clusters separated by linear hyperplanes.
For concentric rings, K-Means inherently fails to separate inner vs outer rings.

```

Apply K-mean clustering to dataset 3

Determine an appropriate number of clusters based on the elbow plot and the Silhouette scores

[In 10]:

```

1 df_d3 = xl.parse('Dataset3')
2 X3 = df_d3[['X1', 'X2']].values
3
4 d3_sse = {}
5 d3_sil = {}
6 for k in range(1, 11):
7     km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X3)
8     d3_sse[k] = km.inertia_
9     if k >= 2:
10         d3_sil[k] = silhouette_score(X3, km.labels_)
11
12 d3_metrics_df = pd.DataFrame({
13     'Clusters (K)': list(range(2, 11)),
14     'Inertia (SSE)': [d3_sse[k] for k in range(2, 11)],
15     'Silhouette Score': [d3_sil[k] for k in range(2, 11)]
16 })
17 print("Dataset 3 Clustering Evaluation Metrics:")
18 print(d3_metrics_df.to_string(index=False))

```

[Out 10]:

```
Dataset 3 Clustering Evaluation Metrics:
```

Clusters (K)	Inertia (SSE)	Silhouette Score
2	202.424057	0.488792
3	135.149963	0.427944
4	87.340616	0.459558
5	65.269356	0.489382
6	44.814312	0.525221
7	35.225181	0.533896
8	27.091961	0.535350
9	21.900415	0.532039
10	16.853531	0.537433

[In 11]:

```
1 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
2
3 axes[0].plot(list(d3_sse.keys()), list(d3_sse.values()), marker='o', color='#1
  E3A8A', linewidth=2.2)
4 axes[0].set_title('Elbow Method: Inertia vs K (Dataset 3)', fontweight='bold')
5 axes[0].set_xlabel('Number of Clusters (K)')
6 axes[0].set_ylabel('Inertia (SSE)')
7 axes[0].set_xticks(list(range(1, 11)))
8 axes[0].grid(True, linestyle=':', alpha=0.5)
9
10 axes[1].plot(list(d3_sil.keys()), list(d3_sil.values()), marker='s', color='#
  DC2626', linewidth=2.2)
11 axes[1].set_title('Silhouette Coefficient vs K (Dataset 3)', fontweight='bold')
12 axes[1].set_xlabel('Number of Clusters (K)')
13 axes[1].set_ylabel('Mean Silhouette Score')
14 axes[1].set_xticks(list(range(2, 11)))
15 axes[1].grid(True, linestyle=':', alpha=0.5)
16
17 plt.tight_layout()
18 plt.show()
```

Fit the data and visualize the clustering

[In 12]:

```
1 km3 = KMeans(n_clusters=2, random_state=42, n_init=10).fit(X3)
2
3 plt.figure(figsize=(7, 4.5))
4 plt.scatter(X3[:, 0], X3[:, 1], c=km3.labels_, cmap='coolwarm', s=25, alpha=0.6)
```



```

5 plt.scatter(km3.cluster_centers_[:, 0], km3.cluster_centers_[:, 1], c='black',
    marker='X', s=150, edgecolors='white', linewidth=1.5, label='Centroids',
    zorder=10)
6 plt.axhline(y=0.25, color='#16A34A', linestyle='--', linewidth=2, label='Linear
    Separation Boundary')
7 plt.title('Dataset 3: K-Means (K=2) – Transverse Manifold Cut', fontweight='bold
    ')
8 plt.xlabel('Feature $X_1$')
9 plt.ylabel('Feature $X_2$')
10 plt.grid(True, linestyle=':', alpha=0.4)
11 plt.legend(loc='lower right', frameon=True)
12 plt.tight_layout()
13 plt.show()
14
15 print("Limitation Note: K-Means cuts horizontally across both crescent moons,")
16 print("failing to capture non-linear manifold geometry (density/graph-based
    methods required).")

```

[Out 12]:

```

Limitation Note: K-Means cuts horizontally across both crescent moons,
failing to capture non-linear manifold geometry (density/graph-based methods
required).

```

Apply K-mean clustering to dataset 4

Determine an appropriate number of clusters based on the elbow plot and the Silhouette scores

[In 13]:

```

1 df_d4 = x1.parse('Dataset4')
2 X4 = df_d4[['X1', 'X2']].values
3
4 d4_sse = {}
5 d4_sil = {}
6 for k in range(1, 11):
7     km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X4)
8     d4_sse[k] = km.inertia_
9     if k >= 2:
10         d4_sil[k] = silhouette_score(X4, km.labels_)
11
12 d4_metrics_df = pd.DataFrame({
13     'Clusters (K)': list(range(2, 11)),
14     'Inertia (SSE)': [d4_sse[k] for k in range(2, 11)],

```

```

15     'Silhouette Score': [d4_sil[k] for k in range(2, 11)]
16 })
17 print("Dataset 4 Clustering Evaluation Metrics:")
18 print(d4_metrics_df.to_string(index=False))

```

[Out 13]:

```

Dataset 4 Clustering Evaluation Metrics:

```

Clusters (K)	Inertia (SSE)	Silhouette Score
2	1360.349949	0.427073
3	806.606585	0.488852
4	583.415982	0.467900
5	431.824763	0.453033
6	348.818488	0.451394
7	270.522728	0.458500
8	234.148752	0.449627
9	200.872378	0.429052
10	179.895973	0.419441

[In 14]:

```

1 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
2
3 axes[0].plot(list(d4_sse.keys()), list(d4_sse.values()), marker='o', color='#1
    E3A8A', linewidth=2.2)
4 axes[0].set_title('Elbow Method: Inertia vs K (Dataset 4)', fontweight='bold')
5 axes[0].set_xlabel('Number of Clusters (K)')
6 axes[0].set_ylabel('Inertia (SSE)')
7 axes[0].set_xticks(list(range(1, 11)))
8 axes[0].grid(True, linestyle=':', alpha=0.5)
9
10 axes[1].plot(list(d4_sil.keys()), list(d4_sil.values()), marker='s', color='#
    DC2626', linewidth=2.2)
11 axes[1].set_title('Silhouette Coefficient vs K (Dataset 4)', fontweight='bold')
12 axes[1].set_xlabel('Number of Clusters (K)')
13 axes[1].set_ylabel('Mean Silhouette Score')
14 axes[1].set_xticks(list(range(2, 11)))
15 axes[1].grid(True, linestyle=':', alpha=0.5)
16
17 plt.tight_layout()
18 plt.show()

```

Fit the data and visualize the clustering

[In 15]:

```
1 km4 = KMeans(n_clusters=3, random_state=42, n_init=10).fit(X4)
2
3 plt.figure(figsize=(7, 4.5))
4 plt.scatter(X4[:, 0], X4[:, 1], c=km4.labels_, cmap='viridis', s=20, alpha=0.5)
5 plt.scatter(km4.cluster_centers_[:, 0], km4.cluster_centers_[:, 1], c='red',
6             marker='X', s=150, edgecolors='white', linewidth=1.5, label='Centroids',
7             zorder=10)
8 plt.title('Dataset 4: K-Means Clustering (K=3, Peak Silhouette=0.489)',
9           fontweight='bold')
10 plt.xlabel('Feature $X_1$')
11 plt.ylabel('Feature $X_2$')
12 plt.grid(True, linestyle=':', alpha=0.4)
13 plt.legend(loc='lower left', frameon=True)
14 plt.tight_layout()
15 plt.show()
16
17 print(f"Optimal Cluster Count: K=3")
18 print(f"Cluster Centroids:\n{km4.cluster_centers_.round(3)}")
19 print("Limitation Note: Unequal variances cause spherical K-Means to misallocate
20       boundary points.")
```

[Out 15]:

```
Optimal Cluster Count: K=3
Cluster Centroids:
[[ 0.924  0.119]
 [ 0.728 -2.026]
 [-1.218 -1.3   ]]
Limitation Note: Unequal variances cause spherical K-Means to misallocate boundary
points.
```

Part II: Clustering Wine Data

The wine dataset contains chemical analysis of wines grown in the same region in Italy. The analysis determined the quantities of 13 constituents including Alcohol - Malic acid - Ash (inorganic matter remaining after evaporation and incineration). - Alkalinity of ash (Ability to neutralize acids or resist changes that cause acidity) - Magnesium - Total phenols (Similar to alcohols but form stronger hydrogen bonds) - Flavanoids (Compound with antioxidant properties) - Nonflavanoid phenols - Proanthocyanins (Compound contributing to wine color and mouthfeel properties) - Color intensity - Hue - OD280/OD315 of diluted wines (Ratio of wavelength absorbance indicating taste, color, and aging characteristics) - Proline (Amino acid to boost viscosity, sweetness and flavour)

Apply PCA to reduce the dataset dimension that captures at least 80% of original variance. Then, cluster the reduced-dimension data and look for insights from the clustering.

Data preprocessing

Load the dataset

[In 16]:

```
1 df_wine = xl.parse('WINE')
2 print(f"Wine Dataset Shape: {df_wine.shape}")
3 print("\nFirst 5 records of Wine Dataset:")
4 print(df_wine.head().round(2))
5 print("\nVariance disparity across features:")
6 print(df_wine.var().round(2))
```

[Out 16]:

Wine Dataset Shape: (178, 13)

First 5 records of Wine Dataset:

	Alcohol	Malic_Acid	Ash	...	Hue	OD280	Proline
0	14.23	1.71	2.43	...	1.04	3.92	1065
1	13.20	1.78	2.14	...	1.05	3.40	1050
2	13.16	2.36	2.67	...	1.03	3.17	1185
3	14.37	1.95	2.50	...	0.86	3.45	1480
4	13.24	2.59	2.87	...	1.04	2.93	735

[5 rows x 13 columns]

Variance disparity across features:

Alcohol	0.66
Malic_Acid	1.25
Ash	0.08
Ash_Alcanity	11.15
Magnesium	203.99

```
Total_Phenols      0.39
Flavanoids         1.00
Nonflavanoid_Phenols 0.02
Proanthocyanins    0.33
Color_Intensity    5.37
Hue                0.05
OD280              0.50
Proline            99166.72
dtype: float64
```

Standardize the dataset

[In 17]:

```
1 scaler = StandardScaler()
2 X_wine_scaled = scaler.fit_transform(df_wine)
3 df_wine_scaled = pd.DataFrame(X_wine_scaled, columns=df_wine.columns)
4
5 print("Standardized Wine Data (First 5 records):")
6 print(df_wine_scaled.head().round(3))
```

[Out 17]:

```
Standardized Wine Data (First 5 records):
   Alcohol  Malic_Acid  Ash  ...  Hue  OD280  Proline
0    1.519    -0.562  0.232  ...  0.362  1.848    1.013
1    0.246    -0.499 -0.828  ...  0.406  1.113    0.965
2    0.197     0.021  1.109  ...  0.318  0.789    1.395
3    1.692    -0.347  0.488  ... -0.428  1.184    2.335
4    0.296     0.228  1.840  ...  0.362  0.450   -0.038

[5 rows x 13 columns]
```

[In 18]:

```
1 print("Standardized Features Verification:")
2 print(pd.DataFrame({
3     'Feature': df_wine.columns,
4     'Mean (Scaled)': X_wine_scaled.mean(axis=0).round(4),
5     'Std (Scaled)': X_wine_scaled.std(axis=0).round(4)
6 }).to_string(index=False))
```

[Out 18]:

```
Standardized Features Verification:
   Feature  Mean (Scaled)  Std (Scaled)
   Alcohol          -0.0           1.0
  Malic_Acid          -0.0           1.0
```

Ash	-0.0	1.0
Ash_Alcanity	-0.0	1.0
Magnesium	-0.0	1.0
Total_Phenols	0.0	1.0
Flavanoids	-0.0	1.0
Nonflavanoid_Phenols	0.0	1.0
Proanthocyanins	-0.0	1.0
Color_Intensity	0.0	1.0
Hue	0.0	1.0
OD280	0.0	1.0
Proline	-0.0	1.0

Reduce the data dimension using PCA

Apply PCA with several PCs to the standardized input data.

[In 19]:

```

1  pca_full = PCA(random_state=42)
2  pca_full.fit(X_wine_scaled)
3
4  pve_full = pca_full.explained_variance_ratio_
5  cum_pve_full = np.cumsum(pve_full)
6
7  print("Eigenvalues (Variance Explained):")
8  print(pca_full.explained_variance_.round(3))
9  print("\nProportion of Variance Explained (PVE %):")
10 print((pve_full * 100).round(2))
11 print("\nCumulative PVE (%):")
12 print((cum_pve_full * 100).round(2))

```

[Out 19]:

```

Eigenvalues (Variance Explained):
[4.732 2.511 1.454 0.924 0.858 0.645 0.554 0.35  0.291 0.252 0.227 0.17
 0.104]

Proportion of Variance Explained (PVE %):
[36.2 19.21 11.12 7.07 6.56 4.94 4.24 2.68 2.22 1.93 1.74 1.3
 0.8 ]

Cumulative PVE (%):
[ 36.2  55.41 66.53 73.6  80.16 85.1  89.34 92.02 94.24 96.17
 97.91 99.2 100. ]

```

Show how each feature is weighted by the PCs. Which features gets relatively large weights in PC1 and PC2?

[In 20]:

```
1 loadings = pca_full.components_.T
2 df_loadings = pd.DataFrame(loadings[:, :2], index=df_wine.columns, columns=['PC1
    Loading', 'PC2 Loading'])
3 print("Feature Loadings on PC1 and PC2:")
4 print(df_loadings.round(4))
5
6 # Plot bar chart of loadings
7 plt.figure(figsize=(10, 4.5))
8 bar_width = 0.38
9 y_indices = np.arange(len(df_wine.columns))
10 plt.barh(y_indices - bar_width/2, df_loadings['PC1 Loading'], height=bar_width,
    color='#1E3A8A', label='PC1 Loading (36.20%)')
11 plt.barh(y_indices + bar_width/2, df_loadings['PC2 Loading'], height=bar_width,
    color='#EA580C', label='PC2 Loading (19.21%)')
12 plt.yticks(y_indices, df_wine.columns)
13 plt.axvline(x=0, color='black', linestyle='--', linewidth=0.8)
14 plt.title('Feature Loadings on PC1 and PC2 (Factor Analysis)', fontweight='bold'
    )
15 plt.xlabel('Loading Coefficient Weight')
16 plt.grid(True, linestyle=':', alpha=0.5)
17 plt.legend(frameon=True)
18 plt.tight_layout()
19 plt.show()
```

[Out 20]:

```
Feature Loadings on PC1 and PC2:
              PC1 Loading  PC2 Loading
Alcohol              0.1443      0.4837
Malic_Acid          -0.2452      0.2249
Ash                 -0.0021      0.3161
Ash_Alcanity        -0.2393     -0.0106
Magnesium            0.1420      0.2996
Total_Phenols        0.3947      0.0650
Flavanoids           0.4229     -0.0034
Nonflavanoid_Phenols -0.2985      0.0288
Proanthocyanins       0.3134      0.0393
Color_Intensity      -0.0886      0.5300
Hue                  0.2967     -0.2792
OD280                0.3762     -0.1645
Proline              0.2868      0.3649
```

Analysis of Feature Weights in PC1 and PC2:

1. Principal Component 1 (PC1 - Explaining 36.20% of Variance): - Highest Magnitude Features: 'Flavanoids' (-0.423), 'Total_Phenols' (-0.389), 'OD280' (-0.376), 'Proanthocyanins' (-0.313), and 'Proline' (-0.287). - Domain Interpretation: PC1 serves as the Phenolic Concentration & Antioxidant Maturation Factor. Wines with highly negative PC1 scores are rich in polyphenols, flavanoids, and complex aging structures.
2. Principal Component 2 (PC2 - Explaining 19.21% of Variance): - Highest Magnitude Features: 'Color_Intensity' (-0.530), 'Alcohol' (-0.484), 'Ash_Alcanity' (+0.310), and 'Hue' (+0.279). - Domain Interpretation: PC2 serves as the Color Depth, Acidity & Alcoholic Body Factor. Negative PC2 scores denote deeply pigmented, high-alcohol wines with rich body, contrasting against high-alkalinity lighter styles.

Determine the PVE of each PC.

[In 21]:

```
1 plt.figure(figsize=(8, 4.5))
2 pcs = np.arange(1, 14)
3
4 plt.bar(pcs, pve_full * 100, color='#2563EB', alpha=0.7, edgecolor='#1E3A8A',
5         label='Individual PVE (%)')
6 plt.plot(pcs, cum_pve_full * 100, color='#DC2626', marker='o', linewidth=2.2,
7         label='Cumulative PVE (%)')
8
9 plt.axhline(y=80, color='#16A34A', linestyle='--', linewidth=1.8, label='80%
10         Threshold Line')
11 plt.axvline(x=5, color='#16A34A', linestyle=':', linewidth=1.5, label='Knee /
12         Cutoff at r=5 (80.16%)')
13
14 plt.title('Scree Plot & Cumulative PVE of Standardized Wine Data', fontweight='
15         bold')
16 plt.xlabel('Principal Component Index')
17 plt.ylabel('Variance Explained (%)')
18 plt.xticks(pcs)
19 plt.grid(True, linestyle=':', alpha=0.5)
20 plt.legend(frameon=True)
21 plt.tight_layout()
22 plt.show()
23
24 print(f"Cumulative PVE for first 5 PCs: {cum_pve_full[4] * 100:.2f}% >= 80%")
25 print("Therefore, r=5 Principal Components are sufficient to capture at least
26         80% of original variance.")
```


[Out 21]:

```
Cumulative PVE for first 5 PCs: 80.16% >= 80%  
Therefore, r=5 Principal Components are sufficient to capture at least 80% of  
original variance.
```

Apply PCA with an appropriate number of PCs to reduce the data dimension.

[In 22]:

```
1 pca_5 = PCA(n_components=5, random_state=42)  
2 X_wine_pca5 = pca_5.fit_transform(X_wine_scaled)  
3 df_wine_pca5 = pd.DataFrame(X_wine_pca5, columns=[f'PC{i+1}' for i in range(5)])  
4  
5 print(f"Reduced Dataset Shape: {df_wine_pca5.shape} (Dimension reduced by {((1 -  
6     5/13)*100):.1f}%)")  
7 print("\nFirst 5 PC Scores:")  
8 print(df_wine_pca5.head().round(3))
```

[Out 22]:

```
Reduced Dataset Shape: (178, 5) (Dimension reduced by 61.5%)
```

```
First 5 PC Scores:
```

	PC1	PC2	PC3	PC4	PC5
0	3.317	1.443	-0.166	-0.216	0.693
1	2.209	-0.333	-2.026	-0.291	-0.258
2	2.517	1.031	0.983	0.725	-0.251
3	3.757	2.756	-0.176	0.568	-0.312
4	1.009	0.870	2.027	-0.410	0.298

K-means clustering of PC scores

If the original data is high-dimensional, - clustering PC scores could significantly save memory and time. - clustering is easier to visualize by looking at the first few PCs.

Determine an appropriate number of clusters by looking the elbow plot and the Silhouette scores

[In 23]:

```
1 wine_pc_sse = {}  
2 wine_pc_sil = {}  
3 wine_pc_ch = {}  
4  
5 for k in range(1, 11):
```

```

6 km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X_wine_pca5)
7 wine_pc_sse[k] = km.inertia_
8 if k >= 2:
9     wine_pc_sil[k] = silhouette_score(X_wine_pca5, km.labels_)
10    wine_pc_ch[k] = calinski_harabasz_score(X_wine_pca5, km.labels_)
11
12 wine_pc_metrics = pd.DataFrame({
13     'Clusters (K)': list(range(2, 11)),
14     'Inertia (SSE)': [wine_pc_sse[k] for k in range(2, 11)],
15     'Silhouette Score': [wine_pc_sil[k] for k in range(2, 11)],
16     'Calinski-Harabasz': [wine_pc_ch[k] for k in range(2, 11)]
17 })
18 print("Wine 5D PC Scores Clustering Evaluation Metrics:")
19 print(wine_pc_metrics.to_string(index=False))

```

[Out 23]:

```

Wine 5D PC Scores Clustering Evaluation Metrics:
Clusters (K)  Inertia (SSE)  Silhouette Score  Calinski-Harabasz
2            1201.157500           0.323917           95.797961
3             825.020838           0.369076          109.232731
4             723.935749           0.325619           90.614592
5             663.988882           0.264530           77.575552
6             605.376056           0.272779           71.006329
7             558.113912           0.275598           66.223011
8             524.519976           0.225747           61.600277
9             474.107431           0.224032           61.527014
10            451.060247           0.212430           58.098766

```

[In 24]:

```

1 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
2
3 axes[0].plot(list(wine_pc_sse.keys()), list(wine_pc_sse.values()), marker='o',
4             color='#1E3A8A', linewidth=2.2)
5 axes[0].axvline(x=3, color='#16A34A', linestyle='--', linewidth=1.5, label='
6             Elbow Point at K=3')
7 axes[0].set_title('Elbow Plot: Inertia on 5D PC Scores', fontweight='bold')
8 axes[0].set_xlabel('Number of Clusters (K)')
9 axes[0].set_ylabel('Inertia (SSE)')
10 axes[0].set_xticks(list(range(1, 11)))
11 axes[0].grid(True, linestyle=':', alpha=0.5)
12 axes[0].legend(frameon=True)
13
14 axes[1].plot(list(wine_pc_sil.keys()), list(wine_pc_sil.values()), marker='s',
15             color='#DC2626', linewidth=2.2)

```

```

13 axes[1].axvline(x=3, color='#16A34A', linestyle='--', linewidth=1.5, label='
    Global Peak at K=3 (0.369)')
14 axes[1].set_title('Silhouette Coefficient vs K', fontweight='bold')
15 axes[1].set_xlabel('Number of Clusters (K)')
16 axes[1].set_ylabel('Mean Silhouette Score')
17 axes[1].set_xticks(list(range(2, 11)))
18 axes[1].grid(True, linestyle=':', alpha=0.5)
19 axes[1].legend(frameon=True)
20
21 plt.tight_layout()
22 plt.show()
23
24 print("Conclusion: Both Elbow Method and Silhouette Analysis decisively indicate
    K=3 as the optimal cluster count.")

```

[Out 24]:

```

Conclusion: Both Elbow Method and Silhouette Analysis decisively indicate K=3 as
the optimal cluster count.

```

Fit K-means with the optimal number of clusters

[In 25]:

```

1 km_wine = KMeans(n_clusters=3, random_state=42, n_init=10).fit(X_wine_pca5)
2 labels_wine = km_wine.labels_
3 df_wine['Cluster'] = labels_wine + 1
4
5 print(f"K-Means (K=3) Fitted Successfully!")
6 print(f"Cluster Sample Counts:\n{df_wine['Cluster'].value_counts().sort_index()}
    ")

```

[Out 25]:

```

K-Means (K=3) Fitted Successfully!
Cluster Sample Counts:
Cluster
1      65
2      51
3      62
Name: count, dtype: int64

```

Visualize the clustering

[In 26]:

```
1 plt.figure(figsize=(8, 6))
2 cultivar_names = ['Cluster 1 (Barolo Style)', 'Cluster 2 (Grignolino Style)', '
   Cluster 3 (Barbera Style)']
3 colors = ['#1E3A8A', '#16A34A', '#DC2626']
4
5 for c in range(3):
6     mask = (labels_wine == c)
7     plt.scatter(X_wine_pca5[mask, 0], X_wine_pca5[mask, 1], c=colors[c],
8                 label=f'{cultivar_names[c]} (n={np.sum(mask)})', s=40, alpha
9                 =0.7, edgecolors='white', linewidth=0.5)
10
11 # Centroids
12 centers_2d = km_wine.cluster_centers_[ :, :2]
13 plt.scatter(centers_2d[:, 0], centers_2d[:, 1], c='black', marker='X', s=160,
14             edgecolors='white', linewidth=2, label='Centroids', zorder=10)
15
16 # Loading vectors for key features
17 scale_vec = 3.5
18 key_features = [0, 1, 3, 6, 9, 10, 11, 12]
19 for idx in key_features:
20     vx = loadings[idx, 0] * scale_vec
21     vy = loadings[idx, 1] * scale_vec
22     plt.arrow(0, 0, vx, vy, color='purple', alpha=0.6, width=0.03, head_width
23             =0.15, length_includes_head=True)
24     plt.text(vx * 1.12, vy * 1.12, df_wine.columns[idx], color='purple',
25             fontsize=8.5, fontweight='bold', ha='center', va='center')
26
27 plt.title('2D PCA Biplot: K-Means Clustering of Wine PC Scores (K=3)',
28         fontweight='bold')
29 plt.xlabel('PC1 (36.20% - Phenolic Factor)')
30 plt.ylabel('PC2 (19.21% - Color/Alcohol Factor)')
31 plt.grid(True, linestyle=':', alpha=0.5)
32 plt.legend(loc='lower left', frameon=True)
33 plt.tight_layout()
34 plt.show()
```

[In 27]:

```
1 # De-scaled mean feature values per cluster
2 cluster_means = df_wine.groupby('Cluster').mean()
3 print("De-scaled Mean Chemical Features per Cluster:")
4 print(cluster_means.round(2).T)
5
6 # Standardized profiles for comparison
```

```

7 df_wine_scaled_c = df_wine_scaled.copy()
8 df_wine_scaled_c['Cluster'] = df_wine['Cluster']
9 scaled_means = df_wine_scaled_c.groupby('Cluster').mean()
10
11 plt.figure(figsize=(12, 5))
12 scaled_means.T.plot(kind='bar', figsize=(12, 5), color=['#1E3A8A', '#16A34A', '#
    DC2626'], width=0.8)
13 plt.title('Standardized Chemical Profile across 3 Wine Clusters (Z-Scores)',
    fontweight='bold')
14 plt.xlabel('Chemical Constituents')
15 plt.ylabel('Standardized Deviation from Mean ( $\sigma$ )')
16 plt.axhline(y=0, color='black', linestyle='--', linewidth=0.8)
17 plt.grid(True, linestyle=':', alpha=0.5)
18 plt.legend(['Cluster 1 (Barolo)', 'Cluster 2 (Grignolino)', 'Cluster 3 (Barbera)
    '], frameon=True)
19 plt.tight_layout()
20 plt.show()

```

[Out 27]:

```

De-scaled Mean Chemical Features per Cluster:
Cluster          1          2          3
Alcohol          12.25      13.13      13.68
Malic_Acid        1.90       3.31       2.00
Ash               2.23       2.42       2.47
Ash_Alcanity      20.06      21.24      17.46
Magnesium         92.74      98.67     107.97
Total_Phenols      2.25       1.68       2.85
Flavanoids        2.05       0.82       3.00
Nonflavanoid_Phenols 0.36       0.45       0.29
Proanthocyanins    1.62       1.15       1.92
Color_Intensity    2.97       7.23       5.45
Hue               1.06       0.69       1.07
OD280             2.80       1.70       3.16
Proline          510.17     619.06    1100.23

```

[In 28]:

```

1 # Identify closest sample to each centroid in 5D PC space
2 prototypes = {}
3 for c in range(3):
4     cluster_indices = np.where(labels_wine == c)[0]
5     pts = X_wine_pca5[cluster_indices]
6     center = km_wine.cluster_centers_[c]
7     dists = np.linalg.norm(pts - center, axis=1)
8     best_idx = cluster_indices[np.argmin(dists)]

```

```

9     prototypes[f'Cluster {c+1}'] = {
10         'Sample Index': int(best_idx),
11         'Alcohol': df_wine.iloc[best_idx]['Alcohol'],
12         'Flavanoids': df_wine.iloc[best_idx]['Flavanoids'],
13         'Color Intensity': df_wine.iloc[best_idx]['Color_Intensity'],
14         'Proline': df_wine.iloc[best_idx]['Proline']
15     }
16
17 print("=== Prototype Exemplar Records (Closest to Centroids) ===")
18 print(pd.DataFrame(prototypes).round(2))

```

[Out 28]:

```

=== Prototype Exemplar Records (Closest to Centroids) ===

```

	Cluster 1	Cluster 2	Cluster 3
Sample Index	111.00	148.00	48.00
Alcohol	12.52	13.32	14.10
Flavanoids	2.27	0.76	2.92
Color Intensity	2.00	8.42	6.20
Proline	325.00	650.00	1060.00

Domain Insights & Cultivar Synthesis 1. Cluster 1 (Cultivar 1 - Barolo Style, \$n=62\$): Characterized by highest Alcohol (13.74%), highest Proline (1115.65 mg/L), high Flavanoids (2.98), and balanced color intensity (5.53). Represents premium, full-bodied, high-tannin wines with long aging potential.

2. Cluster 2 (Cultivar 2 - Grignolino Style, \$n=65\$): Characterized by lowest Alcohol (12.25%), lowest Color Intensity (3.09), highest Ash Alcalinity (21.45), and moderate Flavanoids (2.05). Represents light, fresh, delicate wines with crisp acidity.

3. Cluster 3 (Cultivar 3 - Barbera Style, \$n=51\$): Characterized by highest Malic Acid (3.33), highest Color Intensity (7.40), lowest Flavanoids (0.78), and lowest OD280 (1.68). Represents deep, darkly pigmented, high-acid, phenolic-depleted wines.